

# Neural Network–Accelerated Optimal Transport for Differentiable Particle Filtering

Research Plan and Technical Summary

March 31, 2026

## 1. Problem Statement

A differentiable particle filter requires solving an optimal transport (OT) problem at every resampling step. The standard approach [1] uses entropy-regularized OT via the Sinkhorn algorithm:  $O(LN^2)$  per step ( $L$  iterations,  $N$  particles). This is the computational bottleneck.

**Goal.** Replace Sinkhorn with a learned map that produces the transport plan in a single forward pass, while preserving differentiability with respect to model parameters.

**Constraints.** The state dimension  $d$  may reach 50–100 in production, but a  $d = 5$  proof-of-concept is the first deliverable.

## 2. Key Distinction: Direct Map vs. Neural Operator

These are two different things. Conflating them causes confusion.

|              | Step 1: Direct map                                                    | Step 2: Neural operator                      |
|--------------|-----------------------------------------------------------------------|----------------------------------------------|
| Input/output | $(\mathbf{w}^0, \mathbf{w}^1, \mathbf{X}) \rightarrow \pi$            | $(\rho_0, \rho_1) \rightarrow T$ or $\psi^*$ |
| Domain       | Finite-dim: $\Delta^N \times \Delta^N \times \mathbb{R}^{N \times d}$ | Function spaces: $L^2 \rightarrow L^2$       |
| PDE          | None                                                                  | Monge–Ampère or Schrödinger system           |
| Resolution   | Fixed $N$                                                             | $N$ -invariant (train small, deploy large)   |
| PINN loss    | No (algebraic constraints only)                                       | Yes (PDE residual)                           |

Step 1 is a supervised regression problem. Step 2 upgrades it to a neural operator *only if* resolution invariance across particle counts is needed.

## 3. Step 1: Learn the Discrete Resampling Map

### 3.1 Setup

Run the particle filter with full Sinkhorn resampling. At every resampling step, record:

- **Input:** source weights  $\mathbf{w}^0 \in \Delta^N$ , target weights  $\mathbf{w}^1 \in \Delta^N$ , particle positions  $\mathbf{X} \in \mathbb{R}^{N \times d}$ .
- **Output:** converged dual variables  $\psi^* \in \mathbb{R}^N$  (or the full transport plan  $\pi \in \mathbb{R}^{N \times N}$ ).

Train a neural network  $f_\theta(\mathbf{w}^0, \mathbf{w}^1, \mathbf{X}) \approx \psi^*$  with MSE loss.

### 3.2 Training Data Generation

**Diversity matters more than volume.** Consecutive resampling steps within one trajectory are correlated. To cover the input space:

1. Run many independent filtering trajectories with varied initial conditions, observation sequences, and model parameters.

2. Ensure the training set spans the full range of weight degeneracy—from near-uniform to highly concentrated (low ESS). This is where the transport problem varies most.
3. Save particle positions and weights; the KDE (if needed later) is reconstructed from these.

### 3.3 Iterative Bootstrapping (Dagger-style)

The learned map changes the filter’s behavior, which shifts the distribution of inputs the map encounters. Fix this with iterative refinement:

1. **Round 1.** Run filter with Sinkhorn. Collect  $(\text{input}, \psi^*)$  pairs. Train network  $f_{\theta_1}$ .
2. **Round 2.** Run filter using  $f_{\theta_1}$  for resampling (with Newton correction for safety). The filter explores new regions of state space. Collect new  $(\text{input}, \psi^*)$  pairs from the Newton corrections. Retrain  $f_{\theta_2}$  on the combined dataset.
3. **Round  $k$ .** Repeat. Stop when the distribution of inputs stabilizes (measure via validation loss on held-out trajectories from the previous round).

### 3.4 Data Validity Under Parameter Inference

When model parameters are known, the filter produces well-specified weights and the transport plans are unambiguously good training data. The more interesting case: the parameters are *unknown* and the filter is part of an inference loop (e.g., expectation-maximization, variational inference, or gradient-based maximum likelihood through the differentiable filter).

**Each transport plan is still a correct training example.** The Sinkhorn solver takes  $(\mathbf{w}^0, \mathbf{w}^1, \mathbf{X})$  and returns  $\psi^*$ . This is a pure optimization problem—it does not depend on whether the weights came from a well-specified model. The neural network is learning to approximate Sinkhorn, not to do filtering. Every  $(\text{input}, \psi^*)$  pair is valid regardless of the current parameter estimates.

**What changes is the input distribution.** With poor parameters, the filter produces more degenerate weights (likelihood concentrates on few particles). As parameters improve, weights become more balanced and particles better distributed. Training only on early-inference data risks poor generalization to the later regime.

**This is handled by the bootstrapping loop.** Each round of DAgger-style refinement (Section 3.3) collects transport plans under the current parameter estimates. The combined dataset across rounds covers the full trajectory of the inference process—from misspecified to well-specified. In fact, the inference setting provides *more* input diversity for free: different parameter values during learning produce different  $(\mathbf{w}, \mathbf{X})$  configurations without requiring manual data augmentation.

### 3.5 Architectural Constraints

The network must respect the problem structure:

- **Permutation equivariance:** relabeling particles should relabel the output. Use set-equivariant architectures (DeepSets, Set Transformer).
- **Marginal constraints:** the output  $\pi$  must satisfy  $\sum_j \pi_{ij} = w_i^0$  and  $\sum_i \pi_{ij} = w_j^1$ . Either enforce via a Sinkhorn projection layer on top of the network, or predict dual variables  $\psi$  and reconstruct  $\pi$  analytically (cleaner).

### 3.6 Loss Function

Primary: MSE on dual variables.

$$\mathcal{L}_{\text{sup}} = \|\psi^* - f_{\theta}(\mathbf{w}^0, \mathbf{w}^1, \mathbf{X})\|^2$$

Optional regularizer: marginal constraint violation (linear constraints, well-behaved).

$$\mathcal{L}_{\text{feas}} = \sum_j \left( \int \text{softmax}_j((\hat{\psi}_j - c(\mathbf{x}, \mathbf{y}_j))/\varepsilon) \rho_0(\mathbf{x}) d\mathbf{x} - w_j^1 \right)^2$$

No PDE loss exists in this formulation. This is not a PINN.

## 4. Step 2: Semi-Discrete Neural Operator (if resolution invariance is needed)

### 4.1 Formulation

Smooth one measure via KDE:  $\rho_0(\mathbf{x}) = \sum_i w_i^0 K_h(\mathbf{x} - \mathbf{x}_i)$ . Keep the target discrete:  $\mu_1 = \sum_j w_j^1 \delta_{\mathbf{y}_j}$ . The semi-discrete OT dual is:

$$\max_{\psi} \sum_j w_j^1 \psi_j + \int \min_j \left( \frac{1}{2} |\mathbf{x} - \mathbf{y}_j|^2 - \psi_j \right) \rho_0(\mathbf{x}) d\mathbf{x}$$

The neural operator learns  $\mathcal{F} : (\rho_0, \{\mathbf{y}_j^1\}) \mapsto \psi^* \in \mathbb{R}^N$ . This is a function-to-vector map; DeepONet is the natural architecture (branch net encodes  $\rho_0$ , output layer gives  $\psi^*$ ).

### 4.2 Differentiability Analysis

Three levels of differentiability, each with different properties:

**1. Dual objective  $g(\psi)$ :** differentiable in  $\psi$ . Gradient:  $\partial g / \partial \psi_j = w_j^1 - \int_{L_j(\psi)} \rho_0 d\mathbf{x}$ , where  $L_j(\psi)$  is the Laguerre cell. Hessian well-defined when  $\rho_0$  is absolutely continuous. This is what makes Newton’s method work.

**2. Optimal dual variables  $\psi^*$ :** differentiable in  $(\rho_0, \mathbf{w}^1, \mathbf{y})$  by the implicit function theorem, provided the Hessian of  $g$  at  $\psi^*$  is non-degenerate.

**3. The transport map  $T(\mathbf{x}) = \mathbf{y}_j$  for  $\mathbf{x} \in L_j$ :** *not* differentiable. Piecewise constant with discontinuities at cell boundaries. This breaks backpropagation.

**Fix:** entropy regularization. Replace hard Laguerre cells with soft assignments:

$$\pi_j(\mathbf{x}) = \rho_0(\mathbf{x}) \cdot \frac{\exp((\psi_j - c(\mathbf{x}, \mathbf{y}_j))/\varepsilon)}{\sum_k \exp((\psi_k - c(\mathbf{x}, \mathbf{y}_k))/\varepsilon)}$$

Smooth in all arguments. Resampled position  $\tilde{\mathbf{x}}_i = \sum_j \pi_j(\mathbf{x}_i) \mathbf{y}_j$  is differentiable. The full pipeline—KDE  $\rightarrow$  neural operator  $\rightarrow$  soft assignment  $\rightarrow$  resample—is end-to-end differentiable.

### 4.3 PINN Loss (Available Here)

The Monge–Ampère residual provides a self-supervised loss:

$$\mathcal{L}_{\text{MA}} = \int \left| \det(\nabla^2 \varphi(\mathbf{x})) - \frac{\rho_0(\mathbf{x})}{\rho_1(\nabla \varphi(\mathbf{x}))} \right|^2 d\mathbf{x}$$

This does not require pre-computed solutions. Use it as a regularizer alongside supervised data from Step 1:

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda \mathcal{L}_{\text{MA}}$$

This is the PINO (physics-informed neural operator) paradigm: data anchors the network to correct solutions; the PDE loss enforces consistency between data points.

#### 4.4 Gradient Correctness

The neural operator outputs  $\hat{\psi} \approx \psi^*$ . Backpropagating through the network gives  $\partial\hat{\psi}/\partial\mathbf{w}$ , not the true  $\partial\psi^*/\partial\mathbf{w}$ . With approximation error, these differ.

**Fix:** use the neural operator as a warm start for 2–3 Newton iterations, then differentiate through the iterations (implicit differentiation at the fixed point). This gives correct gradients while keeping the iteration count low.

### 5. Approximation Error Budget

Two sources of error compound in the semi-discrete formulation:

| Source                  | Controlled by           | Behavior                                                          |
|-------------------------|-------------------------|-------------------------------------------------------------------|
| KDE smoothing           | Bandwidth $h$           | $\rightarrow 0$ as $h \rightarrow 0$ , but ill-conditions the PDE |
| Neural operator         | Network capacity + data | Depends on regularity of $\psi^*(\rho_0, \mathbf{w}, \mathbf{y})$ |
| Entropic regularization | $\varepsilon$           | Already present in Sinkhorn baseline                              |

**Key question:** is the KDE error small relative to the entropic regularization error already incurred by Sinkhorn? If yes, the semi-discrete lifting adds negligible additional error and the resolution-invariance payoff is worth it.

### 6. Scaling Considerations ( $d = 50$ – $100$ )

Standard particle filters fail at  $d \geq 20$ : the required particle count grows as  $\exp(d)$  [5]. Variants that work in high dimensions—localized PF [6], equivalent-weights PF [7], particle flow filters—all exploit problem structure (spatial locality, low-rank observations, conditional independence).

The continuous density route (Fokker–Planck + neural operator) also fails for generic  $d = 50$ : representing  $\rho_t \in L^2(\mathbb{R}^{50})$  on any grid is intractable. Neural operators do not break the curse of dimensionality without structural assumptions.

**Practical path forward:** demonstrate the method at  $d = 5$ , where both the discrete map (Step 1) and the semi-discrete operator (Step 2) are feasible. Scaling to  $d = 50+$  requires identifying and exploiting the structure of the specific target problem (low-rank observations, conditional independence, manifold concentration). The  $d = 5$  result establishes that the resampling map is learnable; the structure exploitation is a separate research contribution.

### 7. Action Plan

1. **Baseline implementation** ( $d = 5$ , fixed  $N$ ). Implement a differentiable particle filter with Sinkhorn resampling. Validate on a standard nonlinear filtering benchmark (e.g., Lorenz-63, stochastic volatility). Record wall-clock time per resampling step.

2. **Data collection.** Run 500+ independent trajectories. Save  $(\mathbf{w}^0, \mathbf{w}^1, \mathbf{X}, \psi^*)$  at every resampling step. Stratify by ESS to ensure coverage of degenerate-weight regimes.
3. **Train direct map (Step 1).** Train a Set Transformer on the saved data with MSE loss on  $\psi^*$ . Measure: (a) prediction accuracy  $\|\hat{\psi} - \psi^*\|$ , (b) Sinkhorn iterations saved when  $\hat{\psi}$  is used as warm start, (c) end-to-end filter accuracy (RMSE on state estimates).
4. **Bootstrap iteration.** Run 2–3 rounds of DAgger-style refinement. Measure whether validation loss decreases across rounds (indicates distribution shift was present and corrected).
5. **Differentiability validation.** Train model parameters end-to-end through the learned resampling step. Compare gradient estimates against finite-difference baselines. Verify that Newton correction (2–3 steps) eliminates gradient bias.
6. **Semi-discrete upgrade (Step 2, optional).** If resolution invariance across  $N$  is needed: implement KDE lifting + DeepONet + soft assignment pipeline. Add PINN loss (Monge–Ampère residual or entropic OT optimality conditions) as regularizer. Compare generalization across  $N \in \{50, 100, 200, 500\}$ .
7. **Scaling study.** Increase  $d$  from 5 to 10, 15, 20. Identify where the method degrades and what structural assumptions are needed to go further.

## 8. Key References

### References

- [1] A. Corenflos, J. Thornton, G. Deligiannidis, and A. Doucet. Differentiable particle filtering via entropy-regularized optimal transport. *ICML*, 2021. [arXiv:2102.07850](#).
- [2] S. Chaudhari, S. Pranav, and J. M. F. Moura. GradNetOT: Learning optimal transport maps with GradNets. 2025. [arXiv:2507.13191](#).
- [3] U. Subedi and A. Tewari. Operator learning: A statistical perspective. 2025. [arXiv:2504.03503](#).
- [4] P. K. Jha. From theory to application: A practical introduction to neural operators in scientific computing. 2025. [arXiv:2503.05598](#).
- [5] C. Snyder, T. Bengtsson, P. Bickel, and J. Anderson. Obstacles to high-dimensional particle filtering. *Monthly Weather Review*, 136(12):4629–4640, 2008.
- [6] P. Rebeschini and R. van Handel. Can local particle filters beat the curse of dimensionality? *Annals of Applied Probability*, 25(5):2809–2866, 2015.
- [7] M. Ades and P. J. van Leeuwen. The equivalent-weights particle filter in a high-dimensional system. *Quarterly Journal of the Royal Meteorological Society*, 141(687):484–503, 2015.
- [8] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, and A. Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS Journal of Data Science*, 2024. [arXiv:2111.03794](#).